Module 1: Foundations

Serge Gaspers (K17 506)

Course Admins: cs3821@unsw.edu.au
(Liam, Gerald, Song)

School of Computer Science and Engineering
UNSW Sydney

Term 3, 2025

Table of Contents

1. Time Complexity

2. Revision of Data Structures and Algorithms

3. Proofs

4. Puzzle

Fast and slow algorithms

- You should be familiar with true-ish statements such as:

  Heap sort is faster than bubble sort.

  Linear search is slower than binary search.

- We would like to make such statements more precise.

- We also want to understand when they are wrong, and why it matters.

Best and worst case performance

- The statements on the previous slide are most commonly made in comparing the worst case performance of these algorithms.
- They are actually false in the best case!
- In most problems in this course, we are concerned with worst case performance, so as to be robust to maliciously created (or simply unlucky) instances.
- We will hardly ever discuss the best case.

Average case performance

- In some circumstances, one might accept occasional poor performance as a trade-off for good average performance over all possible inputs (or a large sample of them).

- Analysing the average case or expected performance of an algorithm requires probabilistic methods and is beyond the scope of this course, except where we rely purely on results of this type from prior courses (e.g. hash table operations, quicksort).

Rates of growth

- We prefer to talk in terms of asymptotics, i.e., long-run behaviour.
  - For example, if the size of the input doubles, the function value could (approximately) double, quadruple, etc.
  - A function which quadruples will eventually exceed a function which doubles, regardless of the values for small inputs.
  - We categorise functions (i.e., runtimes, and therefore algorithms) by their asymptotic rate of growth.
- We'll primarily talk about the worst-case performance of algorithms, but the same method of analysis could be applied to the best case or average case performance of an algorithm.

Big-Oh notation

Definition

We say $f(n) \in O(g(n))$ if for large enough n, $f(n)$ is at most a constant multiple of $g(n)$.

Definition (formal)

We say $f(n) \in O(g(n))$ if there are positive integers C and n_0 such that for all $n \geq n_0$, we have that $f(n) \leq C \cdot g(n)$.

- We often see the notation written as $f(n) = O(g(n))$.
- $g(n)$ is said to be an asymptotic upper bound for $f(n)$.
- The rate of growth of function f is no greater than that of g.
- An algorithm whose running time is $f(n)$ scales at least as well as one whose running time is $g(n)$.
- It's true-ish to say that the former algorithm is 'at least as fast as' the latter.

Big-Oh notation

- Useful to (over-)estimate the complexity of a particular algorithm.

- For uncomplicated functions, such as those that typically arise as the running time of an algorithm, we usually only have to consider the dominant term.

Big-Oh notation: Examples

Example 1

Let $f(n) = 100n$. Then $f(n) \in O(n)$, because $f(n)$ is at most 100 times n for large n.

Example 2

Let $f(n) = 2n + 7$. Then $f(n) \in O(n^2)$, because $f(n)$ is at most 1 times n^2 for large n. Note that $f(n) \in O(n)$ is also true in this case.

Example 3

Let $f(n) = 0.001n^3$. Then $f(n) \notin O(n^2)$, because for any constant multiple of n^2, $f(n)$ will eventually exceed it.

Big-Oh notation: Examples

Recall from prior courses that:

- inserting into a binary search tree takes $O(h)$ time in the worst case, where h is the height of the tree, and
- insertion sort runs in $O(n^2)$ time in the worst case, but $O(n)$ in the best case.

Note that these statements are true regardless of:

- the details of how the algorithm is implemented, and
- the hardware used to execute the algorithm.

Both of these issues change the actual running time, but the dominant term of the running time will still be a constant times h, n^2 or n respectively.

Big-Oh and friends

- $f(n) \in O(g(n))$: f grows asymptotically at most as fast as g.
- $f(n) \in \Omega(g(n))$: f grows asymptotically at least as fast as g, i.e., $f(n) \in \Omega(g(n))$ if and only if $g(n) \in O(f(n))$.
- $f(n) \in \Theta(g(n))$: f grows asymptotically as fast as g, i.e., $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$.
- $f(n) \in o(g(n))$: f grows asymptotically strictly slower than g, i.e., $o(g(n)) = O(g(n)) \setminus \Theta(g(n))$.
- $f(n) \in \omega(g(n))$: f grows asymptotically strictly faster than g, i.e., $\omega(g(n)) = \Omega(g(n)) \setminus \Theta(g(n))$.

Fact

If $f_1 \in O(g_1)$ and $f_2 \in O(g_2)$, then $f_1 + f_2 \in O(g_1 + g_2)$.

- The last term $O(g_1 + g_2)$ is often rewritten as $O\left(\max(g_1, g_2)\right)$, since $g_1 + g_2 \leq 2\max(g_1, g_2)$.

- The same property applies if O is replaced by o, Ω, ω or Θ.

Sum property

- This property justifies ignoring non-dominant terms: if f_2 has a lower asymptotic bound than f_1, then the bound on f_1 also applies to $f_1 + f_2$.
- For example, if f_2 is linear but f_1 is quadratic, then $f_1 + f_2$ is also quadratic.
- This is useful for analysing algorithms that have two or more stages executed sequentially.
- If f_1 is the running time of the first stage and f_2 of the second stage, then we can bound each stage and add the bounds, <u>or</u> simply take the most 'expensive' stage.

Product property

Fact

If $f_1 \in O(g_1)$ and $f_2 \in O(g_2)$, then $f_1 \cdot f_2 \in O(g_1 \cdot g_2)$.

- In particular, if $f \in O(g)$ and λ is a constant (i.e., $\lambda \in O(1)$), then $\lambda \cdot f \in O(g)$ also.
- The same property applies if O is replaced by o, Ω, ω or Θ.
- This is useful for analysing algorithms that have two or more nested parts.
- If each execution of the inner part takes f_2 time, and it is executed f_1 many times, then we can bound each part and multiply the bounds.

Table of Contents

1. Time Complexity

2. Revision of Data Structures and Algorithms

3. Proofs

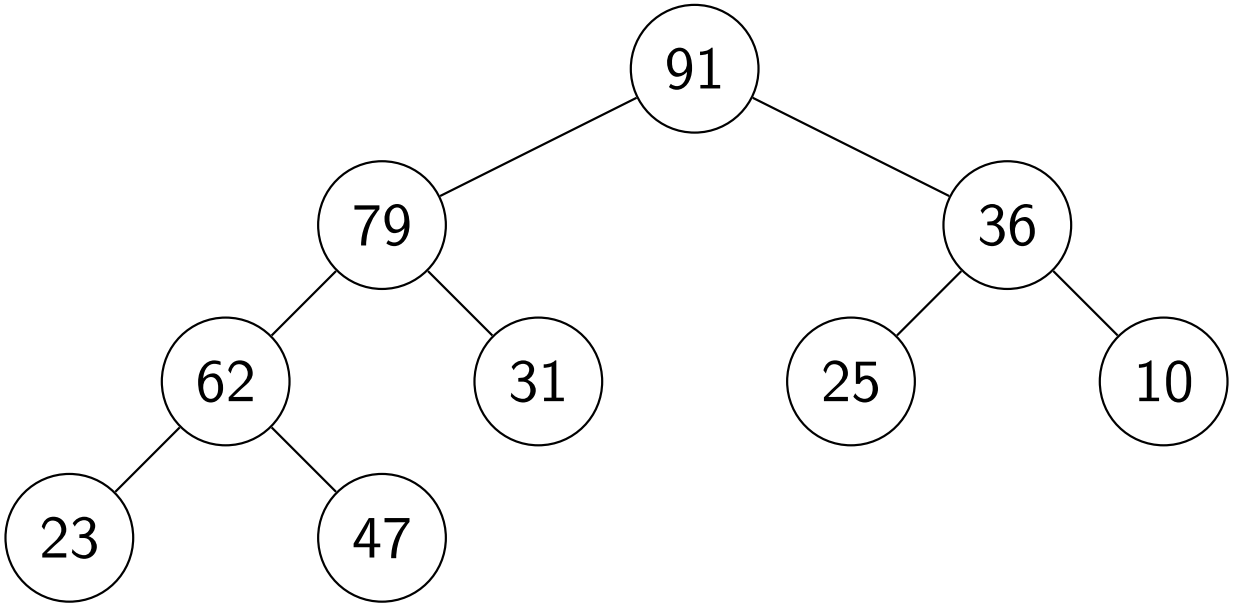
4. Puzzle

(Binary) Heaps

- Store items in a complete binary tree, with every parent comparing $\geq$ all its children.
- This is a max heap; replace $\geq$ with $\leq$ for min heap.
- Used to implement priority queue.

Operations

- Build heap: $O(n)$
- Find maximum: $O(1)$
- Delete maximum: $O(\log n)$
- Insert: $O(\log n)$



i	1	2	3	4	5	6	7	8	9
$A[i]$	91	79	36	62	31	25	10	23	47

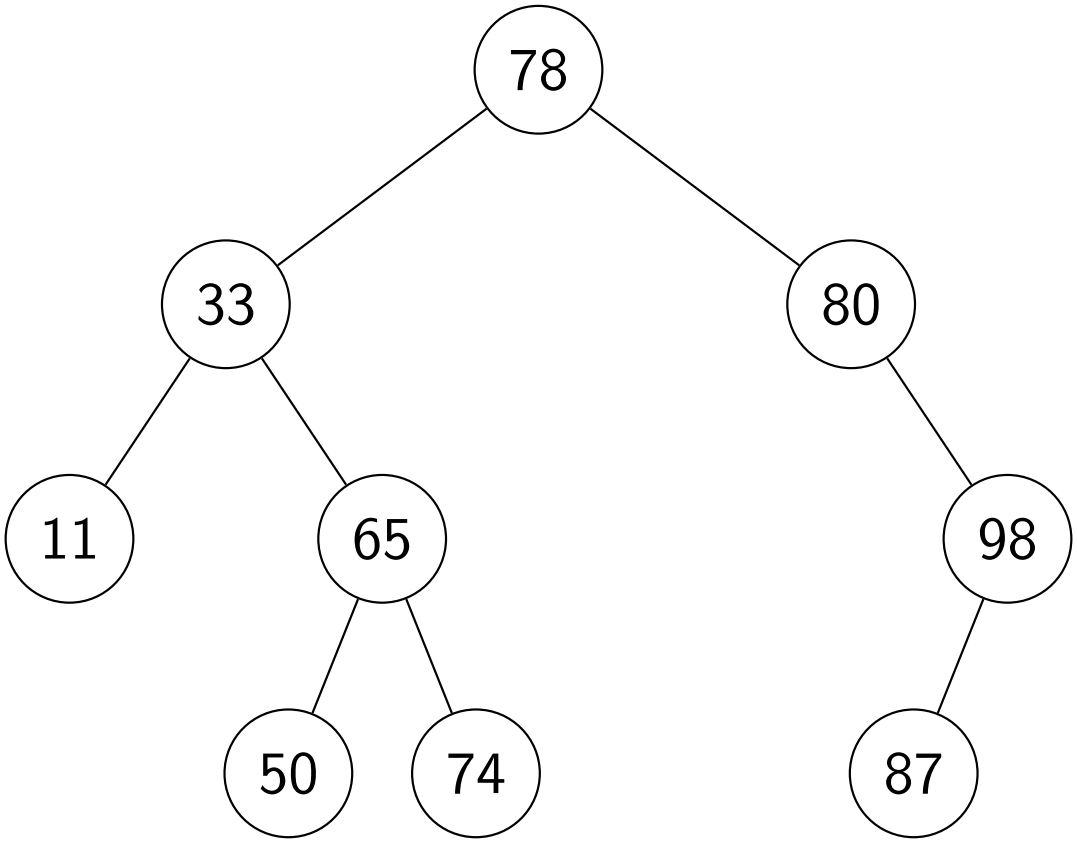
Binary search trees

- Store (comparable) keys or key-value pairs in a binary tree, where each node has at most two children, designated as <u>left</u> and <u>right</u>
- Each node's key compares greater than all keys in its left subtree, and less than all keys in its right subtree.

Operations

Let h be the height of the tree, that is, the length of the longest path from the root to the leaf.

- Search: $O(h)$
- Insert/delete: $O(h)$


78
33
80
11
65
98
50
74
87

Self-balancing binary search trees

- In the best case, $h \approx \log_2 n$. Such trees are said to be balanced.
- In the worst case, $h \approx n$, e.g., if keys were inserted in increasing order.
- Fortunately, there are several ways to make a self-balancing binary search tree (e.g. B-tree, AVL tree, red-black tree).
- Each of these performs rotations to maintain certain invariants, in order to guarantee that $h \in O(\log n)$ and therefore all tree operations run in $O(\log n)$ time.
- Red-black trees are detailed in CLRS, but in this course it is sufficient to write "self-balancing binary search tree" without specifying any particular scheme.

Problem

You are given an array A of $n-1$ elements. Each number from 1 to n appears exactly once, except for one number which is missing. Find the missing number.

Idea

Record whether you've seen each possible value.

- This leads to a linear time, linear space algorithm.

Challenge

How could you solve the problem in linear time and <u>constant</u> space?

Frequency tables

- To count the occurrences of integers from 1 to m, simply make a zero-initialised array of frequencies and increment every time the value is encountered.
- The real question is how to count the occurences of other kinds of items, such as large integers, strings, and so on.
- Try to do the same thing: map these items to integers from 1 to m, and count the occurences of the mapped values.

Hash tables

- Store values indexed by keys.
- Hash function maps keys to indices in a fixed size table.
- Ideally no two keys map to the same index.

Operations (expected)

- Search for the value associated to a given key: $O(1)$
- Update the value associated to a given key: $O(1)$
- Insert/delete: $O(1)$

- A situation where two (or more) keys have the same hash value is called a collision.
- When mapping from a large key space to a small table, it's impossible to completely avoid collisions.
- There are several ways to manage collisions – for example, separate chaining stores a linked list of all colliding key-value pairs at each index of the hash table.

Operations (worst case)

- Search for the value associated to a given key: $O(n)$
- Update the value associated to a given key: $O(n)$
- Insert/delete: $O(n)$

Table of Contents

1. Time Complexity

2. Revision of Data Structures and Algorithms

3. Proofs

4. Puzzle

Propositional logic

- Propositions are true/false statements. Some examples:
  - The sky is orange.
  - Basser Steps consists of more than 100 stairs.
  - Everyone in this room is awake.
- The goal of an argument is usually to establish a relationship between propositions, often also using the following operations.

Notation	Meaning	Formal term
$\neg P$	not P	negation
$P \wedge Q$	P and Q	conjunction
$P \vee Q$	P (inclusive-) or Q	disjunction
$P \rightarrow Q$	if P then Q	implication

- First-order logic adds quantifiers "for all" ($\forall$) and "for some" ($\exists$) over variables.

Proof by induction

- Used to prove a sequence of propositions $P_1, P_2, P_3, \ldots$.
- If the first proposition is true, and each one implies the next, then they are all true.
- If we make an infinite line of dominoes so that each one can knock over the next, and we tip over the first domino, then all the dominoes will fall.
- We also make use of strong induction: if the first proposition is true, and the first k always imply the $(k+1)$th, then they are all true.

Proof by contradiction

- Used to prove a proposition P by considering its negation.
- If the assumption $\neg P$ leads to a contradiction ($Q \wedge \neg Q$ for some proposition Q), then $\neg P$ is impossible, so P holds.
- Example: double-move chess, where each side moves twice on their turn.
  - P: With perfect play, White can at least draw.
  - Suppose otherwise ($\neg P$). Then Black must have a winning strategy that works no matter what White does (Q).
  - White could waste their first turn by moving a knight out and back, then implement the winning strategy themselves.
  - So Black's strategy is not winning after all! ($\neg Q$)

Reasoning about algorithms

- Whenever we present an algorithm, we must justify its correctness and efficiency.
- We need to provide reasoning for the claims that:
  - it always gives the right answer, and
  - it runs in the time complexity we claim.
- Straightforward claims might need only a sentence of reasoning, but for a less obvious claim, the burden is higher.
- Your arguments need to contain enough details to fully convince most students of this class.

Algorithm

We recursively apply the following algorithm to the subarray $A[\ell..r]$.

If $\ell = r$, i.e., the subarray has only one element, we simply exit. This is the base case of our recursion.

Otherwise:

- first define $m = \left\lfloor \frac{\ell+r}{2} \right\rfloor$, the midpoint of the subarray,
- then apply the algorithm recursively to $A[\ell..m]$ and $A[m+1..r]$, and
- finally <u>merge</u> the subarrays $A[\ell..m]$ and $A[m+1..r]$.

Example: mergesort

- The depth of recursion in mergesort is $\log_2 n$.
- On each level of recursion, merging all the intermediate arrays takes $O(n)$ steps in total.
- Thus, mergesort always terminates, and in fact it terminates in $O(n \log_2 n)$ steps.
- Merging two sorted arrays always produces a sorted array, thus, the output of mergesort will be a sorted array.
  - This is actually a proof by induction on the size of the array!
  - Where we can communicate the idea succinctly, we should do so. An argument can be rigorous without notation for its own sake.

The role of proofs in algorithm design

- Sometimes it is <u>not</u> easy to gather from the description of an algorithm:
  - that it terminates, rather than entering an infinite loop,
  - how many steps it takes to terminate, and whether this is acceptably small, or
  - whether the answer it reports is actually correct.
- We always need to justify the claims we make about algorithms, and occasionally that requires a detailed proof.

Stable Matching Problem

- Suppose you have hired n frontend engineers and n backend engineers to build n apps in pairs.
- Every frontend engineer submits a list of preferences, which ranks all the backend engineers from their most preferred partner to least preferred, <u>and vice versa</u>.
- We'd like to make n separate pairs in order to "make everyone happy" in some sense.

Definition

A matching is an assignment of engineers so that no-one belongs to two or more pairs.

Definition

A perfect matching is a matching involving all frontend engineers and all backend engineers, i.e., where everyone belongs to exactly one pair.

Our task is to design a perfect matching that somehow satisfies the frontend and backend engineers, with regards to their preferences.

Question

Can we assign every engineer their first preference partner?

Answer

No, not necessarily!

- We'll have to lower our expectations. We would still like to guarantee some fairness.
- If a frontend engineer and a backend engineer were both very unhappy with their allocated partner, they might quit our company and go build an app together.

Definition

A stable matching is a perfect matching in which there are no two pairs (f_1, b_1) and (f_2, b_2) such that:

- f_1 prefers b_2 to b_1, **and**
- b_2 prefers f_1 to f_2.

Stable matchings are self-enforcing; no unmatched pair of frontend engineer f_1 and backend engineer b_2 will quit.

We will design an algorithm which produces a stable matching.

Stable Matching Problem: Examples

In these examples, we'll label the engineers for ease of reference.

- Frontend: Flora 👩‍💻 and Fred 👨‍💻
- Backend: Betty 👩‍💻 and Bilal 👨🏾‍💻

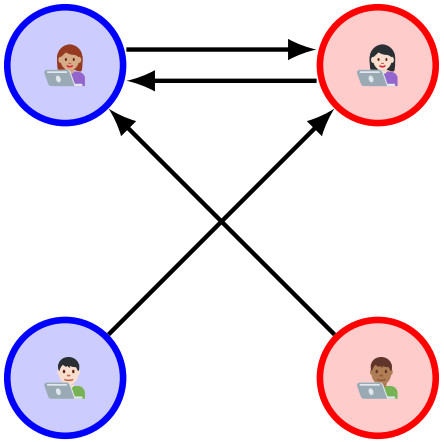
In each example, all four engineers will rank their two counterparts from most preferred to least preferred.

Stable Matching Problem: Example 1

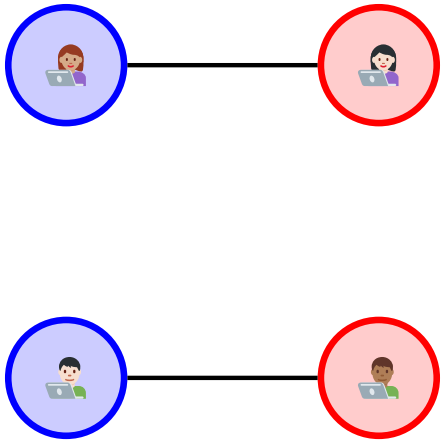
Flora 👩🏽‍💻: Betty 👩🏻‍💻, Bilal 👨🏾‍💻
Fred 👨🏻‍💻: Betty 👩🏻‍💻, Bilal 👨🏾‍💻

Betty 👩🏻‍💻: Flora 👩🏽‍💻, Fred 👨🏻‍💻
Bilal 👨🏾‍💻: Flora 👩🏽‍💻, Fred 👨🏻‍💻

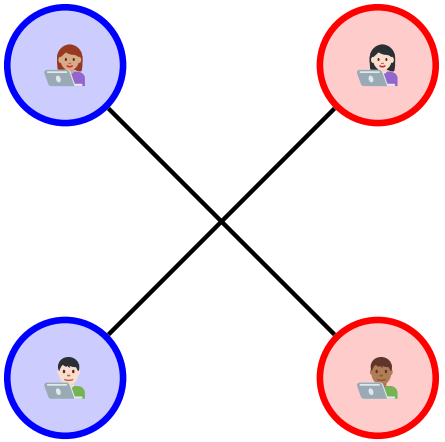
Preferences



Stable



Not stable

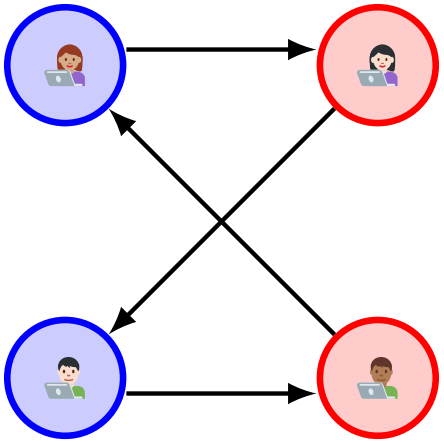


Stable Matching Problem: Example 2

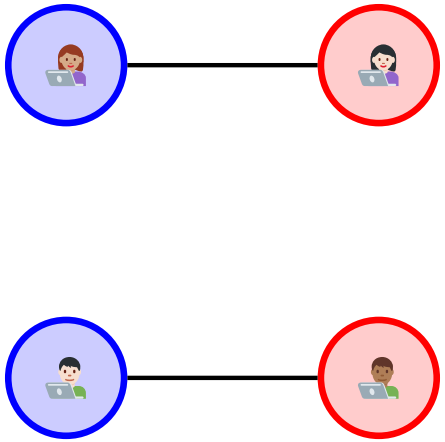
Flora 👩🏽‍💻: Betty 👩🏻‍💻, Bilal 👨🏾‍💻
Fred 👨🏻‍💻: Bilal 👨🏾‍💻, Betty 👩🏻‍💻

Betty 👩🏻‍💻: Fred 👨🏻‍💻, Flora 👩🏽‍💻
Bilal 👨🏾‍💻: Flora 👩🏽‍💻, Fred 👨🏻‍💻

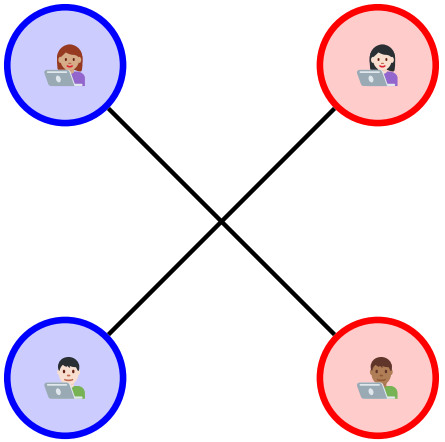
Preferences



Stable



Stable



Question

Given n frontend and n backend engineers, how many ways are there to match them, without regard for preferences?

Answer

$n! \approx (n/e)^n$ - exponentially many in n ($e \approx 2.718$).

Question

Is it true that for every possible collection of n lists of preferences provided by all frontend engineers, and n lists of preferences provided by all backend engineers, a stable matching always exists?

Answer

Perhaps surprisingly, yes!

Question

Can we find a stable matching in a reasonable amount of time?

Answer

Yes, using Gale and Shapley's Deferred Acceptance algorithm.

- Produces pairs in stages, with possible revisions
- Frontend engineers will be pitching to backend engineers (arbitrarily chosen). Backend engineers will decide to accept a pitch or not.
- A frontend engineer who is not currently in a pair will be called solo.
- Start with all frontend engineers solo.

Outline of algorithm:

- While there is a solo frontend engineer who has not pitched to all backend engineers, pick any such solo frontend engineer, say Femi 👩🏾💻.
- Femi pitches to the highest ranking backend engineer Brad 👨🏻💻 on his list, ignoring any who he has pitched to before.
  - If Brad is not yet paired, he accepts Femi's pitch (at least tentatively).
  - Otherwise, if Brad is already paired with someone else, say Frida 👩🏻💻:
    - if Brad prefers Femi to Frida, he turns down Frida and makes a new pair with Femi (making Frida now solo)
    - otherwise, Brad will turn down Femi and stay with Frida.

Question. Does the algorithm work? And is it efficient?

To answer this, we will prove the following 3 claims:

Claim 2

The algorithm produces a perfect matching, i.e., when the algorithm terminates, the $2n$ engineers form n separate pairs.

Claim 3

The matching produced by the algorithm is stable.

Claim 1

The algorithm terminates after $\leq n^2$ rounds.

Claim 1

The algorithm terminates after $\leq n^2$ rounds.

Proof

- In every round of the algorithm, one frontend engineer pitches to one backend engineer.
- Every frontend engineer can make a pitch to a particular backend engineer at most once.
- Thus, every frontend engineer can make at most n pitches.
- There are n frontend engineers, so in total they can make $\leq n^2$ offers.
- Thus there can be no more than n^2 many rounds.

Claim 2

The algorithm produces a perfect matching, i.e. when the algorithm terminates, the $2n$ engineers form n separate pairs.

Proof

- Assume that the algorithm has terminated, but a frontend engineer Felix 👨‍💻 is still solo.
- This means that Felix has already pitched to all the backend engineers.
- A backend engineer is not paired only if no-one has pitched to them. Since Felix has pitched to everyone, all n backend engineers must be paired.
- No frontend engineer can have more than one partner, so all n frontend engineers must also be paired, including Felix.
- This is a contradiction, completing the proof.

Claim 3

The matching produced by the algorithm is stable.

- Recall that each frontend engineer pitches in order from their most preferred to their least preferred backend counterpart.
- Therefore, the sequence of backend engineers paired with a particular frontend engineer is in this order also.
- Each backend engineer is initially not paired, then accepts the first pitch made to them, and only ever switches to a different frontend counterpart who they prefer to their current partner.
- Thus, each backend engineer is paired with frontend engineers in order from their least preferred to their most preferred.

Proof

We will prove that the matching is stable using proof by contradiction.

Assume that the matching is not stable. Thus, there are two pairs:

- Feng 👨‍💻 and Bina 👩‍💻, and
- Farah 👩🏽‍💻 and Bayo 👨🏿‍💻,

such that:

- Feng prefers Bayo over Bina and
- Bayo prefers Feng over Farah.

Proof (continued)

Feng 👨‍💻: . . . , Bayo 👨🏿‍💻, . . . , Bina 👩‍💻, . . .

Bayo 👨🏿‍💻: . . . , Feng 👨‍💻, . . . , Farah 👩🏽‍💻, . . .

- Since Feng prefers Bayo over Bina, he must have made a pitch to Bayo before Bina.
- Bayo must have either:
  - rejected Feng because he was already paired with a frontend engineer he prefers to Feng, or
  - accepted Feng only to later rescind this and accept a pitch from someone he prefers to Feng.

Proof (continued)

Feng 👨‍🦳: . . . , Bayo 🧑🏿‍🦲, . . . , Bina 👩‍🦳, . . .

Bayo 🧑🏿‍🦲: . . . , Feng 👨‍🦳, . . . , Farah 🧑🏾‍🦰, . . .

- In both cases Bayo would become paired with someone higher than Feng in his preferences.
- However he ends up with Farah, who he prefers even less than he does Feng.
- This is a contradiction, completing the proof.

Table of Contents

1. Time Complexity

2. Revision of Data Structures and Algorithms

3. Proofs

4. Puzzle

On a circular highway there are n petrol stations, unevenly spaced, each containing a different quantity of petrol. It is known that the total amount of petrol from all stations is exactly enough to go around the highway once.

You want to start at a petrol station, take all its fuel, and drive clockwise around the highway once. Each time you reach a petrol station, you will take all its fuel too.[1] The only problem is the possibility that you might run out of fuel before reaching the next petrol station!

Prove that there is always at least one starting petrol station to make it around the highway.

[1] Your petrol tank is large enough to hold all the fuel.

That's All, Folks!!